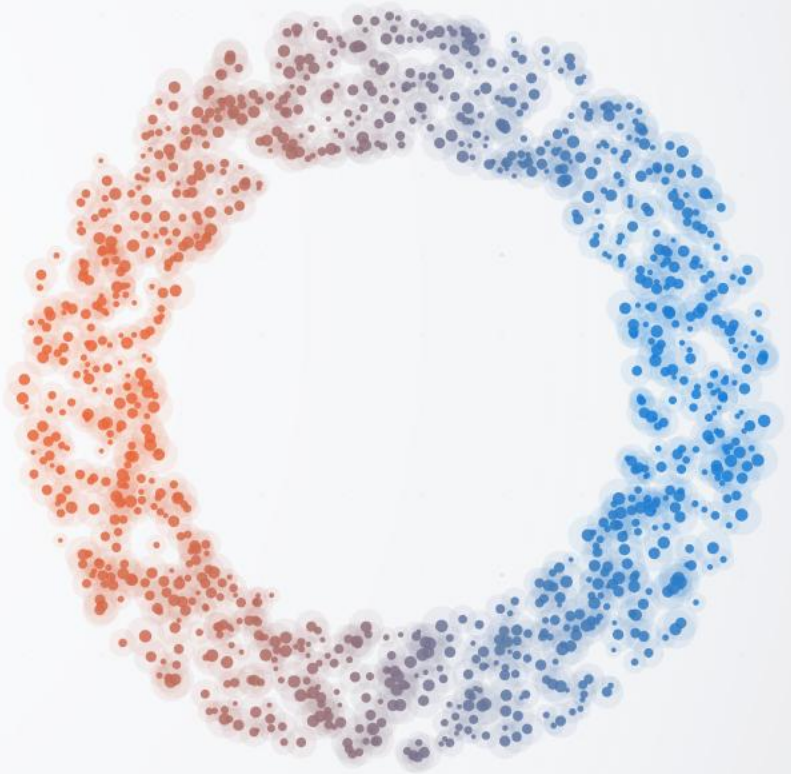
SLIDES

design-harness.slides.yuni.cat

デザインハーネス

をエンジニア向けに説明する

～デザイナーの判断基準を、コードと同じように運用する～



ゆにねこ / AIAU・GDG関西 / 2026/09/25

自己紹介

X
@harineko_univ

SPEAKER

ゆにねこ



- AIAU
- GDG関西 スタッフ

AI で UI を作ると起きること

- デザインシステム・トークン・コンポーネントは完備



AI で UI を作ると起きること

デザインシステム・トークン・
コンポーネントは完備

それでも AI 生成画面は
「それっぽいけれど、**何かが違う**」



AI で UI を作ると起きること

デザインシステム・トークン・
コンポーネントは完備

それでも AI 生成画面は
「それっぽいけれど、**何かが違う**」

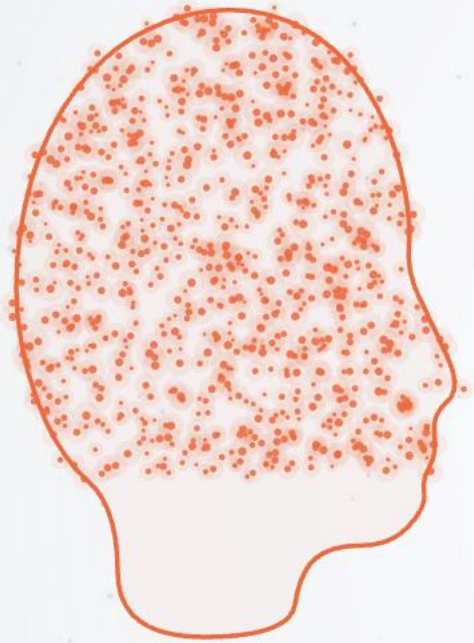
レビューで**繰り返される**同じ指摘



判断基準や
「なぜダメなのか」の理由は、
デザイナーの頭の中にしかない。



判断基準は「個人の頭の中」と「散在した議論」にある



個人の課題　**判断基準の未言語化**

「どれも妥当に見えてしまい、
自分の意見を言い切れない」[N1]

判断基準は「個人の頭の中」と「散在した議論」にある





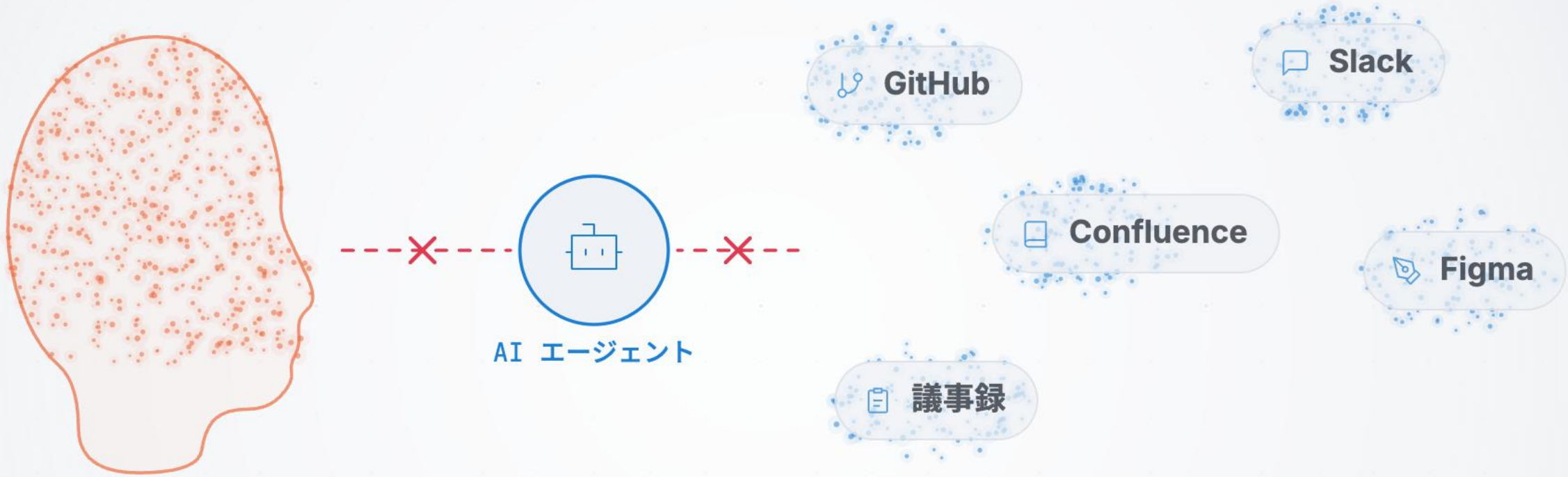
個人の課題　判断基準の未言語化

「どれも妥当に見えてしまい、
自分の意見を言い切れない」[N1]

チームの課題　判断経緯・文脈の散在

「なぜ最小幅 `210px` なのか」を知るのは古参メンバーのみ
[Z1]

判断基準は「個人の頭の中」と「散在した議論」にある



個人の課題　判断基準の未言語化

「どれも妥当に見えてしまい、
自分の意見を言い切れない」[N1]

チームの課題　判断経緯・文脈の散在

「なぜ最小幅 `210px` なのか」を知るのは古参メンバーのみ
[Z1]

どちらも暗黙知のまま → AI から参照できない

エンジニアには見覚えがある構造

かつてのコード開発		いまのデザイン・UI 生成
コーディング規約はシニアの頭の中	≈	判断基準はデザイナーの頭の中
レビューで毎回同じ書き方の指摘	≈	デザインレビューで毎回同じ指摘
「なぜこの実装なのか」は Slack にある	≈	「なぜこの余白なのか」は Slack にある

エンジニアには見覚えがある構造

かつてのコード開発		いまのデザイン・UI 生成
コーディング規約はシニアの頭の中	≈	判断基準はデザイナーの頭の中
レビューで毎回同じ書き方の指摘	≈	デザインレビューで毎回同じ指摘
「なぜこの実装なのか」は Slack にある	≈	「なぜこの余白なのか」は Slack にある
→ linter, CI, ADR で解決		→ 

デザインハーネスとは



```
DESIGN.md · tokens.json · rules/
```

専門職の判断基準を頭の外に出し、コードのように運用する。

書ける　レビューできる　± 差分が残る　全員に配れる

— 橋本 哲勇（LayerX）[P1] p.12 ／ nicotomo さんの要約:「デザイナーの判断基準を頭の外に出し、コード同様に運用する仕組み」[N1]

デザインハーネスとは

ポイントは

~~書き出し~~

ではなく

運用

ドキュメントの増量ではなく、
実参照・自動検査・継続更新の**仕組み化**



「コード同様に運用する」とは

コード運用の仕組み

- バージョン管理（Git）
- ルールへの ID 付与
- lint / 自動テスト
- CI による継続的検証
- ドリフト（乖離）検知
- ADR / PR レビュー

「コード同様に運用する」とは

コード運用の仕組み	デザインハーネスでの対応物
バージョン管理（Git）	`DESIGN.md`・トークン・コンポーネント契約を Git 管理
ルールへの ID 付与	`HTML-BUTTON-TYPE` 等、ルール ID と検査ロジックを紐付け
lint / 自動テスト	UI の自動チェッカー（違反時は `exit 1`）
CI による継続的検証	CLI / MCP / CI パイプラインで共通チェッカーを実行
ドリフト（乖離）検知	再生成した成果物と実ファイルの差分検知（ビルド失敗）
ADR / PR レビュー	判断背景の記録、ルール変更の「提案 → 承認 → 適用」

新しい概念はほとんどない。管理対象が「ソースコード」から「**デザインの判断基準**」に置き換わっただけ。

ハーネスが担う4つの責務

制約

何を使ってよいか、何が必須か

デザイントークン、コンポーネント API、ルール定義

文脈

誰のために、なぜ作るのか

開発ブリーフ、ユースケースシナリオ、設計判断の背景

相互連動

フィードバック

次に何を変えるべきか

不具合修正、ルール・チェックロジック自体の継続的更新

検証

成果物が基準を満たしているか

自動チェックログ、レンダリング結果、人間によるレビュー

単なる4フォルダの作成ではなく、**4責務の相互連動**が肝要 [C1][C2]

ハーネスが担う4つの責務

制約

✓既存のデザインシステムで保証

何を使ってよいか、何が必須か

デザイントークン、コンポーネント API、ルール定義

文脈

✓既存のデザインシステムで保証

誰のために、なぜ作るのか

開発ブリーフ、ユースケースシナリオ、設計判断の背景

相互連動

フィードバック

!不足しがち

次に何を変えるべきか

不具合修正、ルール・チェックロジック自体の継続的更新

検証

!不足しがち

成果物が基準を満たしているか

自動チェックログ、レンダリング結果、人間によるレビュー

「制約」（デザインシステム）は既存でも、「検証」「フィードバック」が不足しがち [C2]

知識の種類を混ぜない



危険: 好みや未決の暫定仕様が、厳格なルールと同列に扱われること

知識の種類を混ぜない

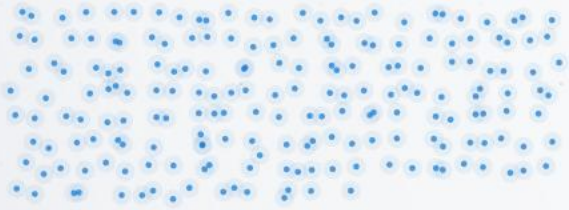
事実

具体例

ボタンの variant は 2 種類

扱い方

実装コードを信頼できる情報源（SSOT）として参照



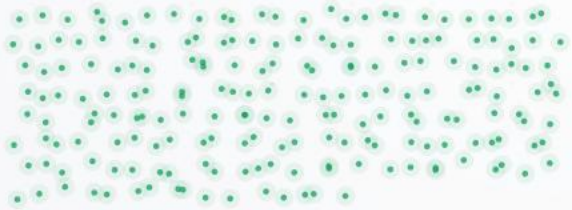
要件

具体例

送信失敗時も入力フォーム値を保持

扱い方

テスト・検査可能な判定条件として定義



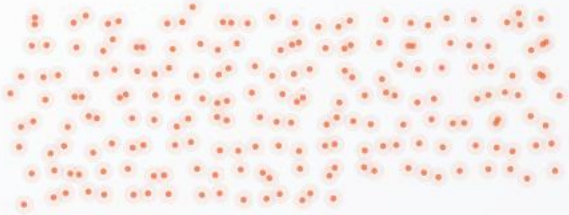
好み

具体例

根拠提示をオファーより前に配置

扱い方

理由と適用スコープをセットで記録



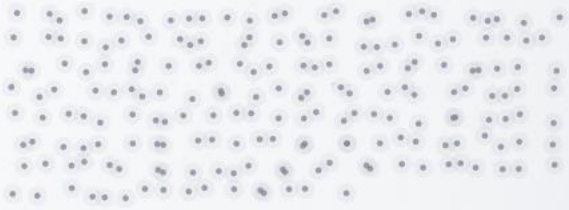
未決

具体例

配信媒体向けブランドルール未策定

扱い方

「未決」と明示し、AI の勝手な推測を遮断



知識の種類を混ぜない

事実	要件	好み	未決
具体例	具体例	具体例	具体例
ボタンの variant は 2 種類	送信失敗時も入力フォーム値を保持	根拠提示をオファーより前に配置	配信媒体向けブランドルール未策定
扱い方	扱い方	扱い方	扱い方
実装コードを信頼できる情報源（SSOT）として参照	テスト・検査可能な判定条件として定義	理由と適用スコープをセットで記録	「未決」と明示し、AI の勝手な推測を遮断

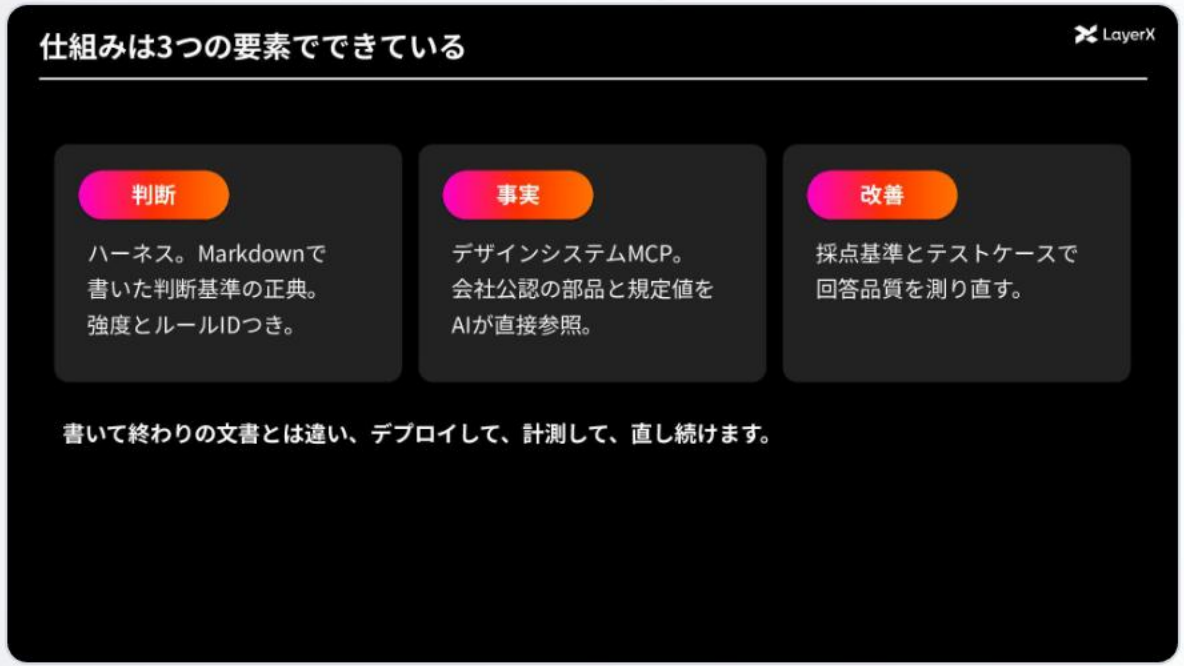
nicotomo さんの個人ハーネスでの実践原則 [N1]

① 実際に下した判断のみを記録

② 迷ったものは「未決」とし、判断材料（実測値・検証結果）とセットで保留

③ 固定の「~~正解集~~」ではなく、検証を重ねる「仮説集」として運用

事例: LayerX のデザインハーネス



出典: 橋本 哲勇（LayerX）「デザインハーネス：専門職の判断基準をコードのように運用する」Bet AI Day 2026, p.13

CASE STUDY — LayerX（バクラク）DesignOps の実践事例 [P1]

判断

↳ 制約・文脈

Markdown で記述した判断基準
（ルール ID・重要度付き）

事実

↳ 制約

デザインシステム MCP
（公式コンポーネント・既定値を AI が直接参照）

改善

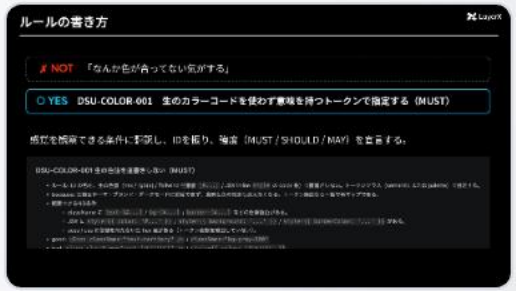
↳ 検証・フィードバック

採点基準とテストケースで、
回答品質を継続的に再測定

事例: 感覚をルールに翻訳する

×「なんか色が合っていない気がする」

デザイナーの感覚のまま —— 観察も再現もできない



出典: 橋本 哲勇（LayerX）Bet AI Day 2026, p.14

事例: 感覚をルールに翻訳する

× 「なんか色が合っていない気がする」

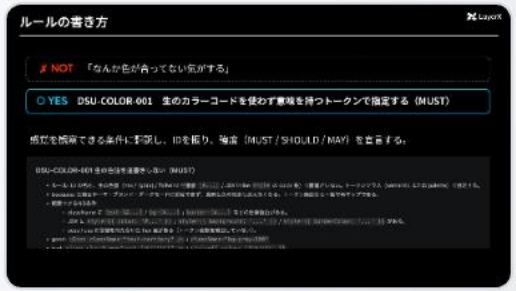
観察できる条件に翻訳

DSU-COLOR-001 MUST

カラーコードのベタ書き禁止、定義トークンで指定

```
color: #3B82F6;  →  color: var(--color-primary);
```

ルール ID を付与 / 重要度 MUST / SHOULD / MAY を明示 / ≒ lint ルールの定義そのもの



出典: 橋本 哲勇（LayerX）Bet AI Day 2026, p.14

事例: 根拠がなければ「評価不可」と答える

AI デザインレビュー結果

色の指定
根拠: `DSU-COLOR-001`
✓ 評価済み

余白の取り方
根拠: 該当ルールなし
評価不可

一般的な UX 論での補完
勝手に隙間を埋めない
しない

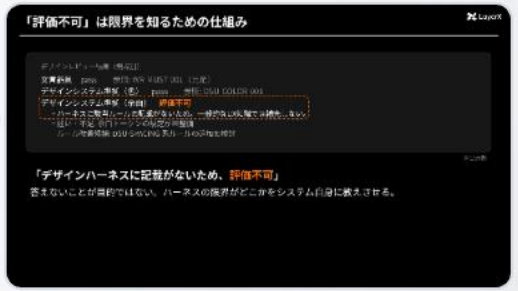
ルール整備のバックログ

不足ルール候補:
余白トークン規定の未整備

該当ルールが無い観点は「評価不可」—— 一般的な UX 論で勝手に補完しない

不足しているルールの候補を同時に提示（例: 余白トークン規定の未整備）

回答の放棄ではなく、ハーネス自身のカバレッジの限界を明示する仕組み

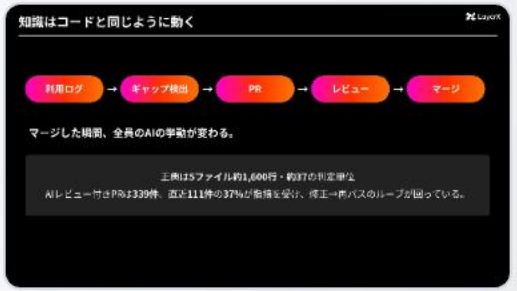


出典: 橋本 哲勇（LayerX）Bet AI Day 2026, p.18

事例: 知識はコードと同じように動く

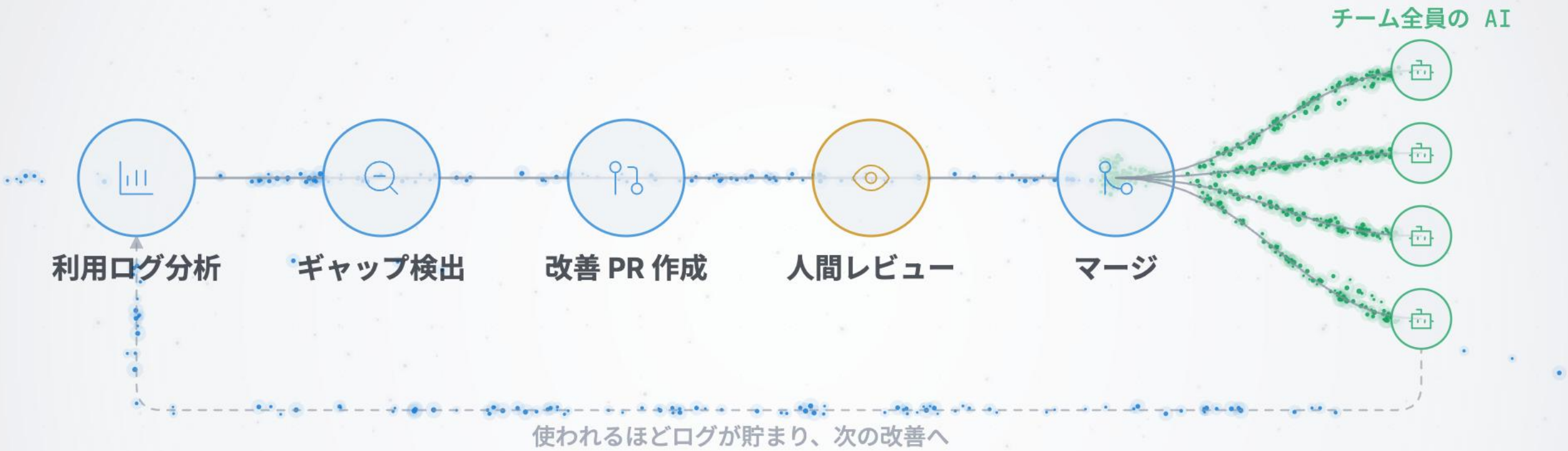


ルールがマージされた瞬間、チーム全員の AI の挙動に反映される



出典: 橋本 哲勇（LayerX）Bet AI Day 2026, p.19

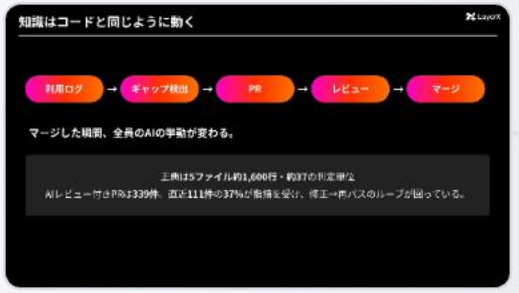
事例: 知識はコードと同じように動く



ルールがマージされた瞬間、チーム全員の AI の挙動に反映される

登壇時点での運用規模 ［P1］

ルール定義	判定単位	AI レビュー付き PR	AI 指摘で修正・再パス
5 ファイル	**約 37**	**339 件**	**37%**
約 1,600 行		累計	直近 111 件中



出典: 橋本 哲勇（LayerX）Bet AI Day 2026, p.19

全体像

人が読むビュー（カタログ・テーマ）

信頼できる情報源（SSOT）



DESIGN.md · tokens
契約 · ルール

カタログ + 依存解決

MCP



知識取得・検査の
インターフェース

Agent Skills



作業手順のパッケージ
（作成 / レビュー / 改善）

成果物

共通チェッカー（CLI / MCP / CI）

レポート



LLM Wiki: 設計判断の経緯・
背景理由のナレッジベース

レポート + 判断の経緯

提案

提案 → 人間の判断 → 更新



承認された更新だけが
情報源へ戻る

承認された更新

SOURCE OF TRUTH

土台: 信頼できる情報源（SSOT）をリポジトリで管理する

DESIGN.md
全体のエントリーポイント
（役割分担・作業手順・実行コマンド）

design/tokens.json
テーマを自動生成
（手動編集禁止・ドリフト検知で不整合を防ぐ）

design/catalog.json › documents
コンポーネント / パターン / シナリオの契約

design/catalog.json › rules
ルール ID・適用スコープ・自動検査ロジック

design/catalog.json › rules[0]

```
{
  "id": "HTML-BUTTON-TYPE",
  "kind": "required-attribute",
  "scope": ["component.button"],
  "selector": "button",
  "attribute": "type",
  "values": ["button", "submit", "reset"]
}
```

定性ルールにも ID を付与し、`manual` として「未評価」を明示

ID と重要度 → レビュー指摘の根拠を追跡できる　／　合致するルールがなければ推測せず「評価不可」を返す [P1]

INTERFACE

MCP: 必要な知識だけを取り出し、共通基準で検査する



- 全ルールを詰め込まず、タスクに必要なコンテキストだけを動的に注入
- CLI と MCP でチェッカーを完全共有 → 検証結果が乖離しない
- 原則読み取り専用（不要な書き込み・任意コマンド実行の権限を持たない）

デザインシステムの API サーバー

WORKFLOW

Agent Skills: 作業手順をパッケージ化する

design-build

画面の新規作成・改修

シナリオ解決
↓ 実装
↓ 自動チェック
↓ 修正
↓ 検証ログ記録

design-review

既存コードのレビュー

- 機械的な違反
- 文脈的な指摘
- 未評価の項目

を分離して報告

design-improve

同じ指摘が頻発したとき

ルール更新案を作成
↓ 承認後に適用
↓ 次のタスクで効果検証

⊘ トークン値や仕様はハードコードしない（「参照先」と「実行コマンド」だけを書く）

↻ 自動修正ループは最大 2 回程度 → 判断が難しければ人間へエスカレーション

Skill = 作業手順書

MCP = 知識の参照と検査のインターフェース

DESIGN.md = それらを束ねる全体インデックス

KNOWLEDGE BASE

LLM Wiki: 判断の「経緯と背景」を育てる

Karpathy 提唱の「Wiki の保守・整理を LLM に委ねる」設計パターン ／ kintone Design System での構成 [Z1]

raw/

一次資料（PR・Issue・議事録...）
人間 / bot が配置・手動編集不可

LLM

wiki/

一次資料を出典に要約・統合
LLM が継続的に保守

`AGENTS.md` 引用ポリシー・運用ルールを定義

Ingest: 一次資料を取り込む

Query: 回答を Wiki に蓄積

Lint: 矛盾・古さを検知

通常の RAG

検索 → 回答 → その場で流れて消える

LLM Wiki

検索 → 回答 → **Wiki ページとして蓄積**
→ 使うほど知識が洗練される

KNOWLEDGE BASE

LLM Wiki: 判断の「経緯と背景」を育てる

Karpathy 提唱の「Wiki の保守・整理を LLM に委ねる」設計パターン ／ kintone Design System での構成 [Z1]

raw/

一次資料（PR・Issue・議事録...）
人間 / bot が配置・手動編集不可

LLM →

wiki/

一次資料を出典に要約・統合
LLM が継続的に保守

`AGENTS.md` 引用ポリシー・運用ルールを定義

Ingest: 一次資料を取り込む

Query: 回答を Wiki に蓄積

Lint: 矛盾・古さを検知

通常の RAG

検索 → 回答 → その場で流れて消える

LLM Wiki

検索 → 回答 → **Wiki ページとして蓄積**
→ 使うほど知識が洗練される

出典の信頼度

- マージ済 PR
- > デザインガイドライン
- > ADR
- > Discussion
- > 議事録

導入効果

「なぜ最小幅 `210px` なのか」

→ 一次資料の出典・時系列付きで、
数分で背景を抽出できる

FEEDBACK LOOP

フィードバック: レビュー指摘を次のタスクへ還元する



- 人間が承認する前の改善提案は、本番の知識ベースへ混入させない
- ルール更新の影響範囲を局所化
  （タスク固有 / コンポーネント / パターン / 全体）
- ログ収集で終わらせず、
  人間のトリアージとルール反映の意思決定が不可欠 [P3]



⇋ 本番ルールの変更には、必ず PR レビューを通す

SKILL

build-design-harness

目的: デザインハーネス・アーキテクチャを手軽に検証・導入する

- 既存資産を調査し、
  再利用できる要素・不足している要素を洗い出す
- 1つの代表的なユースケースで、
  サイクル全体を一気通貫で接続（右図）
- すぐ動かせるスターターキット同梱
  （Node.js 22+・公式 MCP SDK 対応）
- 先行事例（登壇・各社の実践記事・
  公開リポジトリ）を設計に集約
  `docs/references/design-harness.md`



まず「1つのフローを一気通貫で完走させる」ことを最優先に —— 小さく検証してから横展開 [E1-C]

試し方

zsh — your-app

```
# デザインハーネスを導入したいリポジトリで
$ npx skills add mitame-ai/skills --skill build-design-harness
$ claude  # Codex などでも可

> /build-design-harness
  このリポジトリにデザインハーネスを構築して
```

エージェントから実行

Claude Code・Codex などで
スキルを呼び出す
（自然文で頼んでも起動）

MCP 連携

各種クライアントから
`npx design-harness-mcp` で接続

**スキルがリポジトリを調査し、既存のトークン定義やテスト基盤を再利用しながら
1つのフローを一気通貫で構築する。**

結果①:「機械的チェックの通過」と「品質の担保」を区別する

npm run design:check -- examples/profile.html

```
"mechanicalPassed": true,
"checks": [
  { "rule": "HTML-BUTTON-TYPE", "status": "pass" },
  { "rule": "TASK-FIT", "status": "not-evaluated",
    "observation": "Rendered evidence and contextual review are required." }
],
"coverage": { "limits": "Static HTML attributes only. No scripts, CSS, ..." }
```

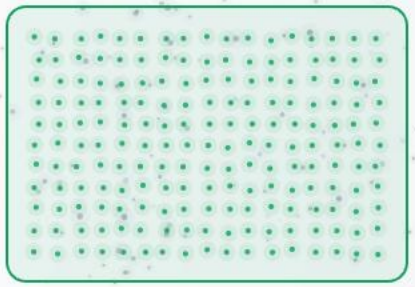
`examples/profile.html` に対するチェック結果（抜粋）

品質の観点（全体）

未検証の観点

目的適合性（TASK-FIT）・
見た目・操作感・文脈の妥当性 ...

機械的に保証できた範囲



- 静的属性のチェックはパスしても、目的適合性（`TASK-FIT`）は「未評価」と正直に記録
- カバレッジの限界（機械で検査できる範囲 / 未検証の観点）を明示
- エージェントの自己申告（「問題なく完成しました」）は鵜呑みにせず、証拠を要求

≒ テストがグリーンでも、テストしていない部分の品質は保証されない

結果②: 検査漏れが新たなルールとして定着するまで

題材: 画像に `alt` 属性がない HTML（実際の実行結果）



結果②: 検査漏れが新たなルールとして定着するまで

題材: 画像に `alt` 属性がない HTML（実際の実行結果）



結果②: 検査漏れが新たなルールとして定着するまで

題材: 画像に `alt` 属性がない HTML（実際の実行結果）



① 「exit 0 で通った」を見過ごさず、改善の契機にする

② 未承認の提案が、勝手にルール化されない安全弁

③ 更新されたルールが、即座に次の検証へ反映される

やってみてわかったこと

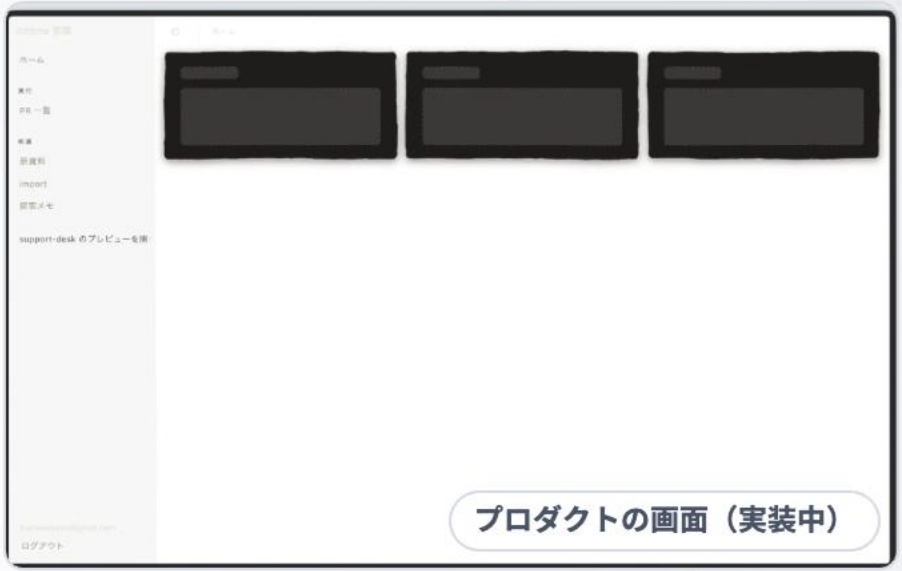
✓ よかったこと

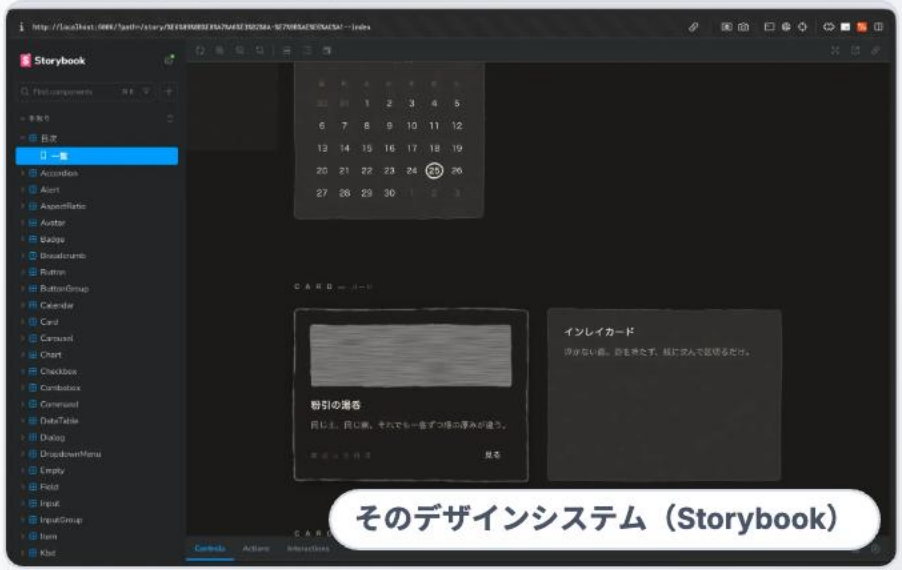
- いろいろな人が提唱するデザインハーネスを、
  自分のアプリのデザインシステムに
  簡単に導入できた

! 課題

- **Storybook だけの場合と比べて、**
  UI が実際に改善したかはまだ確かめられていない
  → 業務委託先でもデザインシステムを構築中。そこでも使ってみる予定

- **知識を**どこに貯めるか**が悩ましい**
  最終的にはデザインシステムの repo に載せたい。でも AI エージェントが
  本来のタスクを離れて別 repo に PR を送るのはしんどい
  → GitHub Issue の発行などで対応したい





まとめ

デザインハーネス＝
デザイナーの判断基準を頭の外に出し、コード同様に運用する仕組み

エンジニアの道具		デザインハーネス
コーディング規約	→	`DESIGN.md` ・コンポーネント契約・ルール ID
lint / CI	→	共通チェッカー（CLI / MCP / CI パイプライン）
開発手順書	→	Agent Skills
ADR / 設計経緯	→	判断記録と LLM Wiki
PR レビュー	→	改善提案 → 人間承認 → 本番適用

まずは身近な1画面、1つのワークフローから。

スターター `build-design-harness` ですぐに検証できます

参考資料

[N1] やしま｜nicotomo「自論に自信がないデザイナーが、『デザインハーネス』を個人で作ってみた」（2026-09-21）
https://note.com/nicotomo_jp/n/n7e5b665316ba

[Z1] saku「散らばった議論を LLM-Wiki でフル活用する AI 時代のデザインシステムのカタチ」（2026-08-05）
https://zenn.dev/cybozu_frontend/articles/llm-wiki-for-design-systems

[P1] 橋本 哲勇 / Tetsuo Hashimoto（LayerX）「デザインハーネス：専門職の判断基準をコードのように運用する」Bet AI Day 2026
https://speakerdeck.com/layerx/bet-ai-day-2026-session05

[C1] Design Harness
https://design-harness.com/

[C2] Kogiso「デザインハーネスとは何か」
https://note.com/kgsi/n/n707d989e1a44

[E1-C] Haruka Shimizu（PKSHA Technology）「1000人規模の組織でデザインハーネスを導入するための第一歩」
https://speakerdeck.com/pkshadeck/1000ren-gui-mo-nozu-zhi-dedezainhanesuwodao-ru-surutamenodi-bu

[E2-B] Sayaka Kubouchi（SoftBank）「Agentic Design Workflowを育てる『三層デザインハーネス』の現在地」
https://speakerdeck.com/sayadesign2/agentic-design-workflow-o-sodateru-sansou-dezain-hanesu-no-genzaichi

[E2-D] Kota Saito（CONCENT）「デザイナーの判断をAIにつなぐ —— 審美眼をハーネスする試み」
https://speakerdeck.com/kotasaito_cnt/dezaina-no-handan-o-ai-ni-tsunagu-shinbi-me-o-hanesu-suru-kokoromi

[P3] Sansan「AIプロトタイピングの精度を上げる—デザインハーネスをチームで育てる方法」
https://note.com/sansan_cpo/n/n0df771f4ef2f

